

हेलो! आपका स्वागत है **Unit 21: JavaScript Runtime & Internal Working** की complete study class में.

मैंने आपके workspace में, unit 21 js runtime and internal working folder के अंदर (directory names mapping matches) ये complete study files write कर दी हैं:

- Study Guide File: chapter21_internal_working.md
- Code Examples File: runtime_examples.js
- Practice Exercises File: runtime_practice.js

चलिए अब Engine compilation phases, Call Stack traces, Memory Heap allocations, Execution Context creation steps, aur Event Loop queues priorities ko simple Hinglish explanation ke sath samajhna shuru karte hain.

PART 1: THE ENGINE ARCHITECTURE

JavaScript execution system: raw code text strings se machine bytecode execution level tak ka engine blueprint.

1. Engine & V8 Compilation Pipeline

What is it? JavaScript Engine ek high-performance runtime system (like Google's **V8**) hai jo script files ko optimize and compile karta hai.

The Flow:

text

Code Text Strings

- Parser scans grammar (Creates AST tree JSON)
 - Interpreter generates bytecodes (Ignition)
 - Optimization compiler runs hot bytecodes directly to machine code (TurboFan)
-

2. Call Stack vs Memory Heap

What is it? Engine local memory split structure:

- **Call Stack:** execution thread coordinates. FILO structure tracking active functions call frames and primitive values (numbers, booleans).
- **Memory Heap:** large unstructured memory space allocating objects, arrays, and functions references.

Simple example

javascript

let count = 10; // directly inside Call Stack

let user = { name: "Ravi" }; // Object inside Memory Heap, pointer address in Call Stack

3. Execution Context

What is it? JavaScript code runs in an active environmental wrapper borders coordinate systems.

- **GEC (Global Execution Context):** root wrapper, runs on start.
- **FEC (Function Execution Context):** generated on each function call.

Two Phases:

1. **Creation Phase:** scans variables declarations (sets them to undefined) and functions (hoisted completely).
 2. **Execution Phase:** parses line-by-line assigning actual values to variables.
-

PART 2: ASYNCHRONOUS COMPONENT CONSTELLATIONS

1. Web APIs

Why browser APIs are handled outside JS Engine? JavaScript is single-threaded. Web APIs like `setTimeout` and `fetch` run asynchronously in background thread layers, preventing blocking of the main execution thread.

2. Event Loop & Queue Priorities

What is it? Coordinate execution pipelines:

- **Microtask Queue:** high priority queue (`Promises .then`, `async/await`).
- **Macrotask Queue:** standard callback queue (`setTimeout`, `setInterval`).

Queue Rule: Event loop checks Microtask Queue and processes all pending callbacks before checking the Macrotask Queue.

Simple example (Console, Promise & Timeout)

javascript

```
console.log("A");  
setTimeout(() => console.log("B"), 0);  
Promise.resolve().then(() => console.log("C"));  
console.log("D");
```

Output sequence: A → D → C (Microtask) → B (Macrotask)

PART 3: GARBAGE COLLECTION

JS engine tracks object variables link references. If an object is no longer reachable from root references scope boundary levels, it is tagged as unreachable garbage. The "Mark-and-Sweep" scanner deletes it automatically. Developers do not call manual free() operations.

QUICK REVISION SUMMARY

- **Compilation Flow:** Text → AST Parsing → Bytecode → Machine code execution.
 - **Call Stack vs Heap:** Stack holds execution frames and primitives. Memory Heap stores objects/arrays pointers references.
 - **Execution Context:** memory scopes are reserved during creation phase; variables are assigned values during execution phase.
 - **Event Loop priority:** Microtask queue (Promises callbacks) takes absolute priority over Macrotask queue (setTimeout).
 - **Garbage Cleanup:** automated mark-and-sweep sweep cycle deletes unreachable memory items.
-

IMPORTANT DEFINITIONS (INTERVIEW-READY)

1. **JIT Compiler (V8):** Just-In-Time compilation tool that compiles JavaScript source code to executable machine code during runtime.
2. **Abstract Syntax Tree (AST):** Hierarchical JSON object tree mapping code syntax structure.
3. **Execution Context:** Scope environment managing variable allocations and executing instructions.
4. **Call Stack Overrun:** RangeError crash happening when function calls nested limit exceeds stack size memory.

5. **Microtask Queue:** High priority execution queue reserved for Promise callback handlers.
-

IMPORTANT INTERVIEW QUESTIONS & ANSWERS

Q1. What is the difference between Call Stack and Memory Heap?

Ans:

- Call Stack is a structured, fast memory space that manages execution frames, function calls, and primitive values.
- Memory Heap is a large, unstructured memory space used to allocate memory dynamically for objects, arrays, and functions.

Q2. Explain JIT Compilation in V8.

Ans: Just-In-Time compilation compiles JavaScript code at runtime. Instead of interpreting line-by-line slowly, V8 compiles code to bytecode (via Ignition interpreter) and optimizes hot paths to native machine code (via TurboFan compiler) for execution.

Q3. What are the two phases of Execution Context?

Ans:

1. Memory Creation Phase: JS scans variables and functions, initializing variables to undefined and hoisting functions.
2. Code Execution Phase: JS executes code line-by-line, assigning actual values.

Q4. Difference between Microtask Queue and Macrotask Queue?

Ans: Microtask Queue holds high-priority callbacks (Promises, async/await). Macrotask Queue holds lower priority callbacks (setTimeout, setInterval). The Event Loop executes all microtasks before processing the next macrotask.

Q5. Predict output:

```
javascript
console.log("Start");

setTimeout(() => console.log("Timer"), 0);

Promise.resolve().then(() => console.log("Promise"));

console.log("End");
```

Ans: "Start" "End" "Promise" "Timer"

Q6. What is a Memory Leak?

Ans: Occurs when variables or objects are no longer needed but are still kept in memory by mistake (like uncleared global variables or event listeners), preventing Garbage Collection.

Q7. Explain how the Event Loop works.

Ans: The Event Loop monitors if the Call Stack is empty. If it is, it first checks the Microtask Queue and executes all available callbacks, then shifts the first callback from the Macrotask Queue to the Call Stack.

Q8. Predict output:

javascript

```
function foo() {  
    return foo();  
}  
  
foo();
```

Ans: RangeError: Maximum call stack size exceeded (Stack overflow).

Q9. Why are Web APIs handled outside the JS Engine?

Ans: Because the JS Engine is strictly single-threaded. Running non-blocking tasks like timers or network calls directly on the main thread would freeze the page.

Q10. What does hoisting do during the creation phase of Execution Context?

Ans: It reserves memory space for variables and functions before code execution, setting variables to undefined and storing full function declarations.

Q11. Predict output:

javascript

```
console.log(myVar);  
  
var myVar = 5;
```

Ans: undefined (due to hoisting in GEC creation phase).

Q12. Predict output:

javascript

```
console.log(myLet);  
  
let myLet = 10;
```

Ans: ReferenceError: Cannot access 'myLet' before initialization (due to Temporal Dead Zone).

Q13. How does the V8 engine optimize performance?

Ans: By using the TurboFan compiler to compile hot (frequently run) bytecodes directly into machine code.

Q14. What does Global Execution Context create by default?

Ans: GEC creates the global object (window in browsers, global in Node.js) and the this binding context.

Q15. Does setTimeout delay guarantee exact timer callback execution?

Ans: No. It specifies the minimum delay. The callback is placed in the Macrotask Queue and can only run after the Call Stack is empty.

Q16. Predict output:

```
javascript
setTimeout(() => console.log("A"), 0);

Promise.resolve().then(() => {
  Promise.resolve().then(() => console.log("B"));
});
```

Ans: "B" "A" (Both microtasks run before the macrotask timeout is processed).

Q17. How does Garbage Collection identify unused objects?

Ans: Using the "Mark-and-Sweep" algorithm. Objects that are unreachable from root nodes (like global scope) are marked as garbage and swept.

Q18. What is the execution stack order of Function Contexts?

Ans: LIFO (Last In First Out). The latest function called is pushed to the top of the stack and executed first.

Q19. Difference between JS Engine and JS Runtime?

Ans: JS Engine compiles and executes code (e.g. V8). JS Runtime wraps the engine and provides external APIs (like window, DOM, setTimeout) to make a complete execution environment (e.g. Chrome browser).

Q20. Can we run code without GEC?

Ans: No. All JavaScript execution starts with a Global Execution Context.

PRACTICE QUESTIONS

Question 1: Call Stack Trace Execution

- **Question:** Draw the Call Stack state step-by-step for:

javascript

```
function first() { second(); }
```

```
function second() { console.log("Run"); }
```

```
first();
```

- **Solution:**
 1. GEC pushes to Stack.
 2. first() FEC pushes to Stack.
 3. second() FEC pushes to Stack.
 4. console.log pushes, prints "Run", pops.
 5. second() pops off stack.
 6. first() pops off stack.
 7. GEC pops off when script terminates.

Question 2: Microtask nesting priority

- **Question:** Predict output of:

javascript

```
setTimeout(() => console.log("Timeout"), 0);
```

```
Promise.resolve().then(() => {
```

```
  console.log("Promise 1");
```

```
  Promise.resolve().then(() => console.log("Promise 2"));
```

```
});
```

- **Output:** Promise 1 Promise 2 Timeout
- **Explanation:** Nesting Promises creates nested microtasks. Event loop processes all microtasks before shifting to the timeout macrotask.

Question 3: Stack Overflow check

- **Question:** Predict stack output check:

javascript

```
let counter = 0;
```

```
function recurse() {
```

```
    counter++;
```

```
    recurse();
```

```
}
```

```
try { recurse(); } catch(e) { console.log(e.name); }
```

- **Output:** RangeError
- **Explanation:** Recursive function calls without exit conditions exhaust Call Stack memory, throwing a RangeError exception.

10:26 PM